

Java Arrays & Strings

10 Practice Questions — Full Explanations & Dry Runs

Companion to the Java Arrays & Strings notes — each question includes the problem, complete working code, a step-by-step dry run table, and the final output.

Prepared by Akshit Sharma

8510968871 | akshubhardwaj231@gmail.com | akshbuilds.tech

Question 1: Reverse an Array In-Place (Two-Pointer)

Problem: Given an array of integers, reverse it in-place without using any extra array. Example input: [10, 20, 30, 40, 50].

Code

```
static void reverseArray(int[] arr) {
    int left = 0, right = arr.length - 1;
    while (left < right) {
        int temp = arr[left];
        arr[left] = arr[right];
        arr[right] = temp;
        left++;
        right--;
    }
}

public static void main(String[] args) {
    int[] arr = {10, 20, 30, 40, 50};
    reverseArray(arr);
    System.out.println(Arrays.toString(arr));
}
```

Explanation

We use two pointers: **left** starting at index 0 and **right** starting at the last index. In each loop iteration we swap `arr[left]` and `arr[right]`, then move left forward and right backward. The loop stops once left is no longer less than right — at that point every pair has been swapped exactly once, which reverses the whole array in $O(n)$ time and $O(1)$ extra space.

Dry Run

Step	left	right	Array State	Action
Start	0	4	[10,20,30,40,50]	left < right -> continue
1	0	4	[50,20,30,40,10]	swap arr[0], arr[4]
	1	3	[50,20,30,40,10]	left++, right--
2	1	3	[50,40,30,20,10]	swap arr[1], arr[3]
	2	2	[50,40,30,20,10]	left++, right--
End	2	2	[50,40,30,20,10]	left == right -> loop stops

Output

[50, 40, 30, 20, 10]

Question 2: Find the Second Largest Element in an Array

Problem: Given [12, 35, 1, 10, 34, 1], find the second largest element in a single pass, without sorting.

Code

```
static int secondLargest(int[] arr) {
    int largest = Integer.MIN_VALUE;
    int second = Integer.MIN_VALUE;
    for (int v : arr) {
        if (v > largest) {
            second = largest;
            largest = v;
        } else if (v > second && v != largest) {
            second = v;
        }
    }
    return second;
}

public static void main(String[] args) {
    int[] arr = {12, 35, 1, 10, 34, 1};
    System.out.println(secondLargest(arr));
}
```

Explanation

We track two running values: **largest** and **second**. For every element, if it beats the current largest, the old largest slides down into second before largest is updated. Otherwise, if it beats second (and isn't equal to largest), it simply replaces second. This finds both values in a single $O(n)$ pass.

Dry Run

v	Condition Taken	largest	second
start	-	MIN	MIN
12	12 > largest	12	MIN
35	35 > largest	35	12
1	1 > second? no	35	12
10	10 > second (12)? no	35	12
34	34 > largest? no; 34 > second? yes	35	34
1	no change	35	34

Output

34

Question 3: Rotate an Array to the Right by K Positions

Problem: Rotate [1, 2, 3, 4, 5, 6, 7] to the right by k = 3, so the last 3 elements move to the front.

Code

```
static void rotateRight(int[] arr, int k) {
    int n = arr.length;
    k = k % n;
    reverse(arr, 0, n - 1); // reverse whole array
    reverse(arr, 0, k - 1); // reverse first k elements
    reverse(arr, k, n - 1); // reverse remaining elements
}

static void reverse(int[] arr, int lo, int hi) {
    while (lo < hi) {
        int t = arr[lo]; arr[lo] = arr[hi]; arr[hi] = t;
        lo++; hi--;
    }
}

public static void main(String[] args) {
    int[] arr = {1, 2, 3, 4, 5, 6, 7};
    rotateRight(arr, 3);
    System.out.println(Arrays.toString(arr));
}
```

Explanation

This is the classic 'reverse three times' trick. Reversing the whole array puts the last k elements at the front, but in the wrong internal order — so we reverse just that front chunk (length k) and just the remaining back chunk (length n-k) to fix their internal order. All three reversals combined equal a right rotation, done in O(n) time and O(1) extra space.

Dry Run

Step	Operation	Array State
0	initial	[1, 2, 3, 4, 5, 6, 7]
1	reverse(0,6) whole array	[7, 6, 5, 4, 3, 2, 1]
2	reverse(0,2) first k=3	[5, 6, 7, 4, 3, 2, 1]
3	reverse(3,6) remaining	[5, 6, 7, 1, 2, 3, 4]

Output

```
[5, 6, 7, 1, 2, 3, 4]
```

Question 4: Check if a String is a Palindrome

Problem: Determine whether "A man a plan a canal Panama" is a palindrome, ignoring case, spaces, and punctuation.

Code

```
static boolean isPalindrome(String s) {
    String clean = s.toLowerCase().replaceAll("[^a-z0-9]", "");
    int left = 0, right = clean.length() - 1;
    while (left < right) {
        if (clean.charAt(left) != clean.charAt(right)) {
            return false;
        }
        left++;
        right--;
    }
    return true;
}

public static void main(String[] args) {
    String s = "A man a plan a canal Panama";
    System.out.println(isPalindrome(s));
}
```

Explanation

First we normalize the string: lowercase everything and strip out anything that isn't a letter or digit using a regex. This gives us `clean = "amanaplanacanalpanama"`. Then we run the same two-pointer technique as reversing an array — comparing characters from both ends moving inward. If every pair matches, the string reads the same forwards and backwards.

Dry Run

left	right	clean[left]	clean[right]	Match?
0	20	a	a	yes
1	19	m	m	yes
2	18	a	a	yes
3	17	n	n	yes
...	...	...	...	... (continues matching)
10	10	left >= right	-	loop ends -> true

Output

true

Question 5: Check if Two Strings are Anagrams

Problem: Check whether "listen" and "silent" are anagrams of each other.

Code

```
static boolean isAnagram(String s1, String s2) {
    if (s1.length() != s2.length()) return false;
    char[] a1 = s1.toCharArray();
    char[] a2 = s2.toCharArray();
    Arrays.sort(a1);
    Arrays.sort(a2);
    return Arrays.equals(a1, a2);
}

public static void main(String[] args) {
    System.out.println(isAnagram("listen", "silent"));
}
```

Explanation

Two strings are anagrams if they contain exactly the same characters, just in a different order. The simplest way to check that is to convert both strings into char arrays, sort each array, and then compare the sorted arrays for equality. If the sorted character sequences match exactly, the original strings must be anagrams.

Dry Run

Step	s1 -> char[]	s2 -> char[]	After Arrays.sort()
1	[l,i,s,t,e,n]	[s,i,l,e,n,t]	-
2 (sort a1)	[e,i,l,n,s,t]	-	sorted
3 (sort a2)	-	[e,i,l,n,s,t]	sorted
4 (compare)	[e,i,l,n,s,t]	[e,i,l,n,s,t]	Arrays.equals -> true

Output

true

Question 6: Find the First Non-Repeating Character

Problem: Find the first character in "swiss" that does not repeat.

Code

```
static char firstNonRepeating(String s) {
    Map<Character, Integer> count = new LinkedHashMap<>();
    for (char c : s.toCharArray()) {
        count.put(c, count.getOrDefault(c, 0) + 1);
    }
    for (char c : s.toCharArray()) {
        if (count.get(c) == 1) return c;
    }
    return '\0'; // not found
}

public static void main(String[] args) {
    System.out.println(firstNonRepeating("swiss"));
}
```

Explanation

This uses two passes. The first pass builds a frequency map of every character using a LinkedHashMap (which preserves insertion order). The second pass walks the string again, in original order, and returns the first character whose count in the map is exactly 1. Using a hashmap for counting makes lookups $O(1)$, so the whole algorithm runs in $O(n)$ time.

Dry Run

Pass	Char	count map after step
1	s	{s:1}
1	w	{s:1, w:1}
1	i	{s:1, w:1, i:1}
1	s	{s:2, w:1, i:1}
1	s	{s:3, w:1, i:1}
2	s -> count 3	skip
2	w -> count 1	return 'w'

Output

w

Question 7: Binary Search on a Sorted Array

Problem: Search for the value 23 in the sorted array [2, 5, 8, 12, 16, 23, 38, 56, 72, 91].

Code

```
static int binarySearch(int[] arr, int target) {
    int lo = 0, hi = arr.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}

public static void main(String[] args) {
    int[] arr = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91};
    System.out.println(binarySearch(arr, 23));
}
```

Explanation

Binary search repeatedly cuts the search range in half. We look at the middle element: if it equals the target we're done; if it's smaller than the target, the target must be in the right half, so we move lo past mid; if it's larger, the target must be in the left half, so we move hi before mid. Because the range halves every iteration, this runs in $O(\log n)$ time instead of $O(n)$ for linear search.

Dry Run

Iteration	lo	hi	mid	arr[mid]	Decision
1	0	9	4	16	16 < 23 -> lo = mid+1 = 5
2	5	9	7	56	56 > 23 -> hi = mid-1 = 6
3	5	6	5	23	match! return mid = 5

Output

5

Question 8: Two Sum on a Sorted Array (Two-Pointer)

Problem: Given the sorted array [1, 3, 4, 6, 8, 11] and target sum 10, find the pair of numbers that add up to the target.

Code

```
static int[] twoSumSorted(int[] arr, int target) {
    int left = 0, right = arr.length - 1;
    while (left < right) {
        int sum = arr[left] + arr[right];
        if (sum == target) {
            return new int[]{arr[left], arr[right]};
        } else if (sum < target) {
            left++;
        } else {
            right--;
        }
    }
    return new int[]{-1, -1}; // not found
}

public static void main(String[] args) {
    int[] arr = {1, 3, 4, 6, 8, 11};
    System.out.println(Arrays.toString(twoSumSorted(arr, 10)));
}
```

Explanation

Because the array is already sorted, we can use two pointers starting at opposite ends. If the sum of the two pointed-to values is too small, the only way to increase it is to move the left pointer right (to a bigger value). If the sum is too big, we move the right pointer left (to a smaller value). This converges on the answer in $O(n)$ time with $O(1)$ extra space — much faster than checking every pair ($O(n^2)$).

Dry Run

left	right	arr[left]	arr[right]	sum	Action
0	5	1	11	12	12 > 10 -> right--
0	4	1	8	9	9 < 10 -> left++
1	4	3	8	11	11 > 10 -> right--
1	3	3	6	9	9 < 10 -> left++
2	3	4	6	10	match! return [4, 6]

Output

[4, 6]

Question 9: Remove Duplicate Characters from a String (Preserve Order)

Problem: Remove duplicate characters from "programming" while keeping the first occurrence of each character in its original order.

Code

```
static String removeDuplicates(String s) {
    StringBuilder sb = new StringBuilder();
    Set<Character> seen = new LinkedHashSet<>();
    for (char c : s.toCharArray()) {
        if (seen.add(c)) { // add() returns false if c was already present
            sb.append(c);
        }
    }
    return sb.toString();
}

public static void main(String[] args) {
    System.out.println(removeDuplicates("programming"));
}
```

Explanation

We use a `LinkedHashSet` named 'seen' purely as a fast membership check. `Set.add(c)` returns true only the first time `c` is added and false on every later duplicate — so we only append to our `StringBuilder` the very first time we encounter each character. Because `String` is immutable, using `StringBuilder` here avoids creating a new `String` object on every character (which would happen with naive `+=` concatenation).

Dry Run

char	seen.add(c)	sb so far
p	true (new)	p
r	true (new)	pr
o	true (new)	pro
g	true (new)	prog
r	false (dup)	prog
a	true (new)	proga
m	true (new)	progam
m	false (dup)	progam
i	true (new)	progami
n	true (new)	progamin
g	false (dup)	progamin

Output

progamin

Question 10: Count Character Frequencies and Find the Most Frequent Character

Problem: Given "mississippi", count how many times each character appears and report the most frequent one.

Code

```
static char mostFrequent(String s) {
    Map<Character, Integer> freq = new HashMap<>();
    for (char c : s.toCharArray()) {
        freq.put(c, freq.getOrDefault(c, 0) + 1);
    }
    char result = s.charAt(0);
    int max = 0;
    for (Map.Entry<Character, Integer> entry : freq.entrySet()) {
        if (entry.getValue() > max) {
            max = entry.getValue();
            result = entry.getKey();
        }
    }
    return result;
}

public static void main(String[] args) {
    System.out.println(mostFrequent("mississippi"));
}
```

Explanation

First pass: build a HashMap where each key is a character and the value is how many times it has appeared so far, using `getOrDefault` to handle first-time characters cleanly. Second pass: scan the completed map's entries and keep track of whichever character has the highest count seen so far. For "mississippi", both 'i' and 's' appear 4 times each, but 'i' is typically returned first depending on HashMap iteration order in this case.

Dry Run

Character	Final Count in freq map
m	1
i	4
s	4
p	2

Output

i (count = 4; 's' also has count 4 – either is a valid 'most frequent' answer)

Prepared by Akshit Sharma

8510968871 | akshubhardwaj231@gmail.com | akshbuilds.tech